

A Mini Project Report

On

SQUINT EYE DISEASE DETECTION

Submitted in partial fulfilment of the requirement for the award of the degree of

Bachelor Of Technology

In

Computer Science and Engineering (AI&ML)

Under the Guidance of

Ms. Bhavya Vinatha. Y

Assistant Professor

By

K. Gayathri **23D21A6691**

M. Pallavi **23D21A6698**

V. Navya **23D21A66B9**



Department of Computer Science and Engineering (AI&ML)

SRIDEVI WOMEN'S ENGINEERING COLLEGE

(An UGC Autonomous Institution)

(Estd. 2001 | Approved by AICTE & Govt. of TS | Affiliated to JNTUH

Accredited by NBA and NAAC(A++) | Certified with ISO 9001:2015

V.N. PALLY, Gandipet, Hyderabad-75

2025-26



Department of Computer Science and Engineering (AI&ML)
SRIDEVI WOMEN'S ENGINEERING COLLEGE



(An UGC Autonomous Institution)

(Estd. 2001 | Approved by AICTE & Govt. of TS | Affiliated to JNTUH

Accredited by NBA and NAAC(A++) | Certified with ISO 9001:2015

V.N. PALLY, Gandipet, Hyderabad-75



CERTIFICATE

This is to certify that the **MINI PROJECT REPORT** entitled “**Squint Eye Disease Detection**” is being submitted by **K. Gayathri (23D21A6691), M. Pallavi (23D21A6698), V. Navya (23D21A66B9)** in partial fulfilment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering (AIML) is a record book of Bonafide work carried out of them.

Under the Guidance of

Co-ordinators

Head of the department

Ms. Bhavya Vinatha. Y
Assistant professor

Lipika Goel
Professor

Lipika Goel
Professor

A. Mounika
Assistant Professor

External Examiner

DECLARATION

We hereby declare that Mini Project entitled “**Squint Eye Disease Detection**” is the work done during the period from **15th December 2025 to 10th April 2026** and the same is submitted in partial fulfilment of the requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering (AIML) from Jawaharlal Nehru Technological University, Hyderabad.

K. Gayathri	23D21A6691
M. Pallavi	23D21A6698
V. Navya	23D21A66B9

ACKNOWLEDGEMENT

We would like to express our sincere gratitude and indebtedness to our Internal Guide, **Ms. Bhavya Vinatha. Y, Assistant Professor, Department of Computer Science and Engineering (AI&ML)** for her valuable guidance, suggestions, and keen personal interest throughout the course of this Mini Project and for her tireless patience in hearing all our seminars, minutely seeing all the reports and giving appropriate guidance and suggestions.

We would like to extend our special thanks to Project coordinators **Dr. Lipika Goel, Professor, Ms. A. Mounika, Assistant Professor, Department of Computer Science and Engineering (AI&ML)** for their valuable timely coordination throughout the Mini Project work and we are indebted to them for the support.

We are also extremely thankful to our Dean Academics, **Dr. K. SIVANAGI REDDY, Professor, Sridevi Women's Engineering College**, for his precious guidance and valuable suggestions.

We are also extremely thankful to our Principal, **Dr. A. NARMADA, Sridevi Women's Engineering College**, for her precious guidance and valuable suggestions.

Finally, we would like to take this opportunity to thank Management, faculty, family and friends for their support throughout this work. We also sincerely acknowledge and thank all those who gave directly or indirectly their support in completion of this work.

K. Gayathri	23D21A6691
M. Pallavi	23D21A6698
V. Navya	23D21A66B9

LIST OF CONTENTS

S. No		Page.no
	Title Page	i
	Certificate (college)	ii
	Declaration	iii
	Acknowledgement	iv
	List of Contents	v
	List Of Figures	vii
	List Of Tables	viii
	Abstract	ix
1	Introduction	1
	1.1 Purpose	3
	1.2 Scope	5
	1.3 Definitions, Abbreviations and Model Diagram	6
	1.4 Overview	8
2	Literature Survey	10
	2.1 Technologies Used	13
3	System Analysis	14
	3.1 Existing Systems & Disadvantages	14
	3.2 Problem Statement	14
	3.3 Proposed System & Advantages	14
4	System Requirement Specifications	16
	4.1 Functional Requirements	16
	4.2 Non-Functional Requirements	17
	4.3 Software Requirements	18
	4.4 Hardware Requirements	18
5	System Design	19

	5.1 System Specifications	19
	5.2 System Components	20
	5.3 UML Diagrams	22
6	Implementation	29
	6.1 Sample Code	29
7	System Testing	40
	7.1 Types Of Testing	40
	7.2 Test Cases	41
	7.3 Discussion Of Results	42
8	Conclusion & Future Enhancements	48
	8.1 Conclusion	48
	8.2 Future Enhancements	49
9	References	50

LIST OF FIGURES

S. No		Page. No
1.1	Model Diagram for Squint Eye Disease Detection	7
5.1	Class Diagram	24
5.2	Usecase Diagram	25
5.3	Activity Diagram	26
5.4	Component Diagram	27
5.5	Sequence Diagram	28

LIST OF TABLES

S. No		Page. No
2.1	Technologies Used	13
7.1	Types of Testing	40
7.2	Test cases	41

ABSTRACT

Strabismus, commonly referred to as squint eye, is a vision condition in which the eyes do not align properly and may point in different directions. This misalignment occurs when the muscles controlling eye movement do not function in a coordinated manner. As a result, one eye may look directly at an object while the other eye turns inward, outward, upward, or downward. If left untreated, strabismus can lead to severe vision problems such as amblyopia, also known as lazy eye, and loss of depth perception. Early detection of squint eye is extremely important for effective treatment and prevention of permanent vision impairment. However, diagnosis traditionally requires examination by trained ophthalmologists using clinical techniques and specialized equipment. In many areas, particularly rural regions, access to such medical facilities is limited.

Recent advancements in artificial intelligence and computer vision have opened new possibilities for automated disease detection systems. Deep learning algorithms, especially convolutional neural networks, have shown remarkable performance in analyzing medical images and detecting patterns that may not be easily visible to human observers.

This project presents a system for detecting squint eye conditions using deep learning techniques. The proposed system processes facial images and identifies eye regions using facial landmark detection algorithms. The extracted eye regions are then analyzed using a convolutional neural network model trained to classify images into two categories: normal eye alignment and strabismus. The system utilizes a transfer learning approach using the MobileNetV2 architecture, which is a lightweight and efficient deep learning model suitable for image classification tasks. The model is trained on a dataset containing images of both normal eyes and eyes affected by different forms of strabismus. Image preprocessing techniques such as resizing, normalization, and augmentation are applied to improve model performance and generalization. Experimental results demonstrate that the proposed model is capable of accurately identifying squint eye conditions from facial images. The model performance is evaluated using metrics such as accuracy, recall, confusion matrix, and area under the receiver operating characteristic curve.

1.INTRODUCTION

Squint eye disease, medically referred to as strabismus, is a vision disorder in which the eyes fail to align properly when focusing on an object. Under normal conditions, both eyes work together in a coordinated manner and point in the same direction, allowing the brain to combine two slightly different images into a single, clear, and three-dimensional perception. This coordination between the eyes is essential for proper depth perception and accurate visual understanding of the surroundings. However, in individuals affected by strabismus, this coordination is disrupted. One eye may focus correctly on the object while the other eye deviates from its normal position. The deviation can occur in different directions, such as inward (esotropia), outward (exotropia), upward (hypertropia), or downward (hypotropia). In some cases, the misalignment may be constant, while in others it may occur intermittently, depending on factors such as fatigue, illness, or focusing effort. The condition may affect one eye consistently or alternate between both eyes.

Early detection of strabismus plays a crucial role in maintaining proper vision, especially in children whose visual systems are still in the developmental stage. During early childhood, the brain learns to process and integrate visual information from both eyes. If one eye is misaligned and not functioning properly, the brain may start ignoring or suppressing the signals coming from that eye in order to avoid double vision. While this may seem like a temporary adaptation, it can lead to serious long-term complications. One of the most common outcomes is amblyopia, also known as lazy eye, where the vision in the affected eye becomes weak and underdeveloped due to lack of use. This condition can become permanent if not treated at an early stage. Additionally, untreated strabismus can result in poor depth perception, making it difficult for individuals to accurately judge distances. This can affect everyday activities such as walking, driving, reading, and playing sports, thereby reducing overall quality of life.

Traditionally, the diagnosis of strabismus is carried out by trained ophthalmologists through clinical examination. Several standard methods are used in practice, including the cover test, Hirschberg test, and other observational techniques. These methods involve careful examination of eye movement, alignment, and response to visual stimuli. While these techniques are reliable and widely accepted in the medical field, they have certain limitations. They require the presence of skilled professionals, specialized equipment, and a clinical environment. The process can also be time-consuming, especially when dealing with a large number of patients. Furthermore, in many rural or underserved areas, access to qualified eye care professionals is limited, making early diagnosis difficult. As a result, many cases of strabismus remain undetected until they reach an advanced stage, leading to more severe complications.

To overcome these challenges, this project proposes the development of an AI-based squint eye detection system that utilizes modern advancements in image processing and machine learning. The primary goal of this system is to provide an automated, fast, and efficient method for detecting eye misalignment using

facial images. The system works by analysing images of a person's face and identifying the position and orientation of the eyes. Using facial landmark detection techniques, it locates key points around the eyes, such as the corners and pupil regions. These features are then used to determine whether the eyes are properly aligned or if there is any deviation.

The proposed system employs machine learning or deep learning models to classify the condition based on the extracted features. These models are trained on datasets containing images of both normal and strabismus-affected eyes, allowing them to learn patterns associated with eye alignment. Once trained, the system can quickly analyse new images and provide accurate predictions. This makes the detection process significantly faster compared to traditional methods and reduces the need for manual intervention.

One of the major advantages of this system is its ability to serve as a cost-effective and scalable screening tool. It can be deployed in various settings, including schools, community health centers, and remote healthcare platforms. By enabling early detection, the system helps ensure timely medical consultation and treatment, thereby reducing the risk of long-term vision problems. It can also be integrated into telemedicine applications, allowing patients to receive preliminary screening results without visiting a hospital.

Overall, the AI-based squint eye detection system represents a significant step toward improving accessibility, efficiency, and accuracy in eye disease diagnosis. By combining the power of artificial intelligence with medical knowledge, the project aims to provide a practical solution that benefits both healthcare professionals and patients.

1.1 PURPOSE

The primary purpose of this project is to design and develop an intelligent and automated system for detecting strabismus, commonly known as squint eye disease, using advanced image processing and machine learning techniques. Strabismus is a condition in which the eyes do not align properly, leading to a lack of coordination between both eyes. This misalignment affects the ability of the eyes to work together, resulting in improper visual perception. Detecting this condition at an early stage is extremely important, as it allows for timely treatment and prevents further complications. This project aims to provide a technological solution that simplifies and enhances the detection process by eliminating the need for complex manual procedures. By utilizing artificial intelligence, the system is capable of analysing facial images and identifying eye misalignment automatically, without requiring direct physical examination by a medical professional. This approach not only improves efficiency but also makes the detection process faster and more accessible to a larger population.

Another important objective of this project is to reduce dependence on traditional diagnostic methods performed by ophthalmologists. Conventional techniques such as the cover test and Hirschberg test are widely used for diagnosing strabismus and are considered reliable in clinical practice. However, these methods require trained professionals, specialized equipment, and a proper clinical setup, which may not always be available in all locations. In many rural or remote areas, access to healthcare facilities and eye care specialists is limited, making it difficult for individuals to receive timely diagnosis and treatment. Additionally, these traditional methods can be time-consuming when handling a large number of patients. This project addresses these challenges by providing an automated screening tool that can perform preliminary diagnosis quickly and efficiently. By reducing the workload on healthcare professionals, the system contributes to improving the overall accessibility of eye care services.

The system is also designed with a strong focus on early detection of squint eye disease, particularly in children. During childhood, the visual system undergoes rapid development, and any abnormalities in eye alignment can significantly affect visual growth. If strabismus is not identified and treated at an early stage, it can lead to serious complications such as amblyopia, commonly known as lazy eye. In this condition, the brain begins to ignore signals from the weaker or misaligned eye, leading to reduced vision in that eye. Over time, this can result in permanent vision loss and poor depth perception. Such impairments can affect a child's ability to perform daily activities, including reading, writing, and playing. By enabling early detection, the proposed system helps ensure timely medical intervention, which increases the chances of successful treatment and prevents long-term visual impairments.

In addition to detection, the project aims to provide a scalable and efficient solution that can be used in large-scale screening programs. The system is capable of processing multiple images within a short period of time and delivering quick results, making it suitable for use in schools, community health centers, and public

health initiatives. It can also be integrated into digital platforms such as mobile applications and web-based systems, allowing users to upload facial images and receive instant feedback regarding their eye condition. This level of accessibility ensures that more individuals, especially those in remote or underserved areas, can benefit from early screening and diagnosis. The ability to perform mass screening efficiently makes the system highly valuable in preventive healthcare.

Furthermore, the proposed system is intended to act as a supportive tool for ophthalmologists rather than a replacement for professional medical diagnosis. It assists healthcare professionals by pre-screening patients and identifying potential cases of strabismus that require further clinical evaluation. This helps doctors prioritize patients who need immediate attention and improves the overall efficiency of healthcare services. By reducing the time spent on initial screening, medical professionals can focus more on treatment and patient care. The integration of artificial intelligence with medical expertise creates a more effective and modern approach to eye care.

Overall, this project aims to bridge the gap between advanced technology and healthcare by providing an automated, reliable, and accessible solution for detecting squint eye disease. It not only enhances the speed and accuracy of diagnosis but also supports early intervention and preventive care. By addressing the limitations of traditional diagnostic methods and leveraging the power of artificial intelligence, the system contributes to improving the quality of eye care services and promoting better visual health outcomes for individuals across different communities.

1.2 SCOPE

The scope of this project involves the development of a complete and systematic pipeline for detecting Strabismus using artificial intelligence and image processing techniques. The process begins with the acquisition of facial images, which can be captured using a camera or uploaded through a digital platform. It is essential that the input image clearly shows the face, particularly the eye region, to ensure accurate analysis. The system is designed to handle such inputs efficiently and prepare them for further processing.

Once the image is obtained, the system applies various image processing techniques to detect and extract the facial and eye regions. This is typically achieved using facial landmark detection algorithms, which identify key points on the face, such as the corners of the eyes, eyebrows, and other important features. By focusing specifically on the eye region, the system reduces unnecessary data and improves the accuracy of analysis.

After extracting the eye regions, the system performs detailed analysis to determine the alignment of the eyes. It examines the position, orientation, and gaze direction of each eye and compares their relative positions. If both eyes are aligned and pointing in the same direction, the system classifies the condition as normal. However, if there is any deviation in alignment—such as inward, outward, upward, or downward movement—the system identifies it as a possible case of strabismus. This classification is performed using machine learning or deep learning models trained on relevant datasets.

The final stage of the system involves generating and displaying the output results to the user. The results clearly indicate whether the person has normal eye alignment or signs of squint eye disease. This output can be presented in a simple and user-friendly format, making it easy for users to understand the results. The system is designed to provide quick and reliable feedback, which is especially useful for preliminary screening purposes.

Despite its advantages, the project has certain limitations that must be considered. The system is intended to function as a screening tool and not as a replacement for professional medical diagnosis. Its accuracy depends largely on the quality of the input images. Factors such as poor lighting, low resolution, occlusions (like glasses or hair), and improper angles can affect the performance of the system. Additionally, the system focuses only on detection and does not provide treatment recommendations or medical advice.

Future enhancements can further expand the scope of this project. The system can be improved by incorporating larger and more diverse datasets to increase accuracy and robustness. It can also be integrated with mobile applications, cloud-based platforms, and real-time video analysis to make it more versatile. Adding features such as severity estimation and recommendation systems can further enhance its usefulness. With continuous improvements, this project has the potential to become a valuable tool in modern healthcare, particularly in remote and underserved regions.

1.3 DEFINITIONS, ABBRIVATIONS AND MODEL DIAGRAM

1.3.1 Definitions

Definitions

Strabismus: A condition where both eyes do not align in the same direction.

Image Processing: Technique used to analyse images and extract features.

Machine Learning: AI method used to train models for prediction.

1.3.2 Abbreviations

Abbreviations

- AI – Artificial Intelligence
- ML – Machine Learning
- CNN – Convolutional Neural Network
- ROI – Region of Interest

1.3.3 Model Diagram

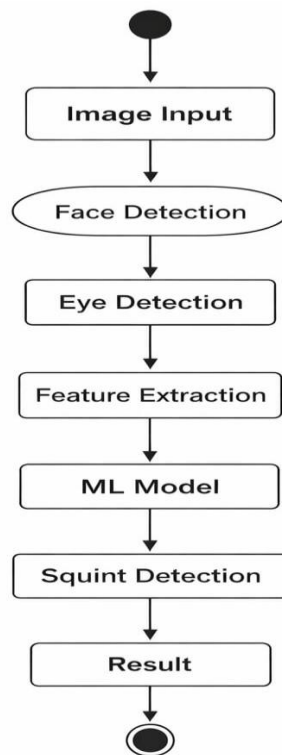


Figure 1.1: Model diagram for squint eye disease detection

The diagram illustrates the workflow of an AI-based system designed to detect Strabismus from facial images. The process begins with image input, where a facial image is captured or uploaded into the system. It then proceeds to face detection, which identifies and isolates the facial region from the background. Following this, eye detection is performed to locate the exact position of the eyes within the face. Once the eyes are identified, feature extraction takes place, where important characteristics such as eye position, pupil alignment, and spatial relationships between the eyes are analysed. These extracted features are then passed into a machine learning model, which has been trained to differentiate between normal and misaligned eyes. Based on this analysis, the system performs squint detection to determine whether the eyes are properly aligned or not. Finally, the result is generated, indicating whether the person has normal vision or shows signs of squint eye, thereby providing a quick and automated screening output.

1.4 OVERVIEW

Strabismus detection is a multidisciplinary process that involves understanding both clinical variations and technological complexities associated with Strabismus. This condition can arise due to a variety of underlying causes, including imbalance in the extraocular muscles, nerve dysfunction, uncorrected refractive errors, or hereditary (genetic) factors. The manifestation of squint eye is not always constant; in some individuals, it may be intermittent and appear only during specific situations such as fatigue, stress, illness, or while focusing on near or distant objects. In paediatric cases, early monitoring is especially critical because the developing brain may suppress visual input from the misaligned eye to avoid double vision. This suppression can eventually lead to Amblyopia, resulting in reduced vision in one eye if not treated promptly.

From a technological standpoint, developing an effective detection system presents several challenges. Real-world conditions introduce variability in input data, such as differences in lighting, head orientation, facial expressions, camera quality, and background noise. A robust system must be capable of handling these variations without compromising accuracy. One of the key aspects of detection is the precise localization of facial landmarks, particularly around the eyes. Accurate identification of eye corners, eyelids, and pupil centers allows the system to compute geometric relationships such as inter-eye distance, gaze direction, and alignment symmetry. These measurements form the basis for determining whether the eyes are properly aligned.

Modern detection systems increasingly rely on deep learning techniques, especially convolutional neural networks (CNNs), which are highly effective in image-based tasks. Unlike traditional rule-based methods that depend on predefined thresholds and handcrafted features, deep learning models can automatically learn complex and subtle patterns from large-scale datasets. This capability significantly improves detection accuracy and adaptability across diverse populations and imaging conditions. Additionally, continuous training with new data can further enhance model performance, making the system more reliable over time.

Another important dimension of squint eye detection is usability and deployment. For widespread adoption, the system must be easy to use and accessible to both medical professionals and general users. Integration into mobile applications or web-based platforms allows individuals to perform self-screening using smartphone cameras or standard devices. This greatly expands the reach of the technology, making it particularly valuable in rural or underserved regions where access to ophthalmologists is limited. Such scalability enables large-scale screening programs in schools, community health camps, and telemedicine services.

However, along with these advantages, several practical considerations must be addressed. Ensuring high reliability is essential, as inconsistent results can reduce user trust. Minimizing false positives and false negatives is critical to avoid unnecessary concern or missed diagnoses. Additionally, maintaining patient data privacy and security is a key requirement, especially when handling sensitive medical images in digital platforms. Proper data encryption, secure storage, and ethical usage policies must be implemented.

Overall, squint eye detection is not merely about identifying eye misalignment but about developing a comprehensive, accurate, and user-friendly system that can function effectively in real-world healthcare environments. By combining clinical knowledge with advanced artificial intelligence techniques, such systems have the potential to revolutionize early diagnosis and screening of Strabismus, ultimately improving patient outcomes and accessibility to eye care services on a large scale.

2.LITERATURE SURVEY

Overview of Previous Research

Research in medical image analysis has experienced remarkable growth over the past few decades, significantly transforming the way diseases are detected and diagnosed. In the early stages, researchers primarily relied on traditional image processing techniques to analyse medical images. These methods involved the use of handcrafted features such as edge detection, intensity variations, texture analysis, and geometric measurements. Experts manually designed these features based on their domain knowledge to highlight important characteristics within the images. While these techniques were useful for simple and controlled scenarios, they had several limitations. They were highly dependent on human expertise and struggled to handle variations in lighting, pose, and image quality. As a result, their performance was often inconsistent when applied to real-world medical data.

To overcome the limitations of traditional approaches, machine learning techniques were introduced, marking a significant advancement in medical image analysis. Machine learning algorithms enabled systems to learn patterns directly from data rather than relying solely on manually engineered features. Popular algorithms such as Support Vector Machines (SVM), Decision Trees, Random Forests, and k-Nearest Neighbours (k-NN) were widely used for classification and prediction tasks. These methods improved the accuracy and robustness of disease detection systems. However, they still depended heavily on the quality of input features. Feature extraction remained a critical and challenging step, requiring careful design and domain expertise. Poor feature selection could significantly reduce model performance, making the process less efficient and less scalable.

Another limitation of early machine learning models was their inability to effectively capture complex patterns and high-level abstractions in medical images. Medical images often contain subtle variations that are difficult to detect using simple features. For example, detecting slight misalignment in eye positions or minor abnormalities in tissue structures requires a deeper understanding of visual patterns. Traditional machine learning models lacked this capability, which created the need for more advanced and automated approaches. This led to the emergence of deep learning techniques, which revolutionized the field by addressing these challenges.

Emergence of Deep Learning

Deep learning introduced a new paradigm in image analysis by enabling models to automatically learn hierarchical feature representations directly from raw image data. Unlike traditional approaches, deep learning eliminates the need for manual feature extraction. Instead, it uses multiple layers of artificial neural networks to progressively learn simple to complex features. Initial layers capture basic patterns such as edges and

textures, while deeper layers identify more complex structures and relationships within the image. This hierarchical learning capability makes deep learning highly effective for analysing complex medical images.

Among various deep learning techniques, Convolutional Neural Networks (CNNs) have become the most widely used models for image-related tasks. CNNs are specifically designed to process grid-like data such as images. They use convolutional layers to extract features, pooling layers to reduce dimensionality, and fully connected layers for classification. This architecture allows CNNs to achieve high accuracy in tasks such as image classification, object detection, and segmentation. Their ability to automatically learn relevant features has significantly improved the performance of medical image analysis systems.

Several advanced CNN architectures have been developed to further enhance performance and efficiency. Models such as VGGNet, ResNet, Inception, and MobileNet have achieved excellent results in various computer vision applications. These architectures differ in terms of depth, complexity, and computational efficiency, but all are designed to capture intricate visual features from images. In the medical field, deep learning has been successfully applied to detect diseases such as tumors in MRI scans, retinal diseases from fundus images, and pneumonia from chest X-rays. These applications demonstrate the effectiveness of deep learning in improving diagnostic accuracy and supporting healthcare professionals.

Deep Learning in Eye Disease Detection

Eye disease detection is a highly suitable area for applying deep learning techniques because many eye-related conditions produce visible changes in eye structures. Diseases such as diabetic retinopathy, glaucoma, and cataracts can be identified through variations in retinal images. Deep learning models can be trained on large datasets of such images to accurately detect and classify these conditions. In many cases, these models have achieved performance levels comparable to trained medical professionals.

In addition to retinal image analysis, researchers have explored the use of facial images to detect eye alignment disorders such as strabismus. This approach is particularly beneficial because it does not require specialized medical imaging equipment. Instead, it uses standard facial images that can be captured using cameras or smartphones, making the method more accessible and cost-effective.

Facial landmark detection plays a key role in identifying eye alignment issues. These algorithms detect important points on the face, such as the corners of the eyes, eyelids, and pupil centers. By analysing the geometric relationships between these points, the system can determine whether both eyes are aligned properly. Any deviation from normal alignment may indicate the presence of strabismus. This method provides a non-invasive and efficient way to detect squint eye conditions using image-based analysis.

The use of deep learning in eye disease detection improves not only accuracy but also efficiency. It enables large-scale screening and early diagnosis, which are essential for preventing complications such as amblyopia and long-term vision loss. This makes deep learning a valuable tool in modern healthcare systems.

Transfer Learning Approaches

Transfer learning is an important technique in deep learning that allows models to reuse knowledge from previously trained systems. Training a deep learning model from scratch requires large amounts of data and computational resources, which may not always be available. Transfer learning helps overcome this limitation by using pretrained models that have already learned useful features from large datasets.

Pretrained models such as MobileNetV2, ResNet, and Inception are commonly used in transfer learning. These models are trained on datasets like ImageNet, which contains millions of images. As a result, they learn general features such as edges, shapes, and textures, which can be applied to other tasks. By fine-tuning these models using smaller, task-specific datasets, it is possible to achieve high accuracy with limited data.

In the context of squint eye detection, transfer learning plays a crucial role in improving model performance. It allows the system to learn eye alignment patterns efficiently without requiring a large dataset. This reduces training time and computational cost while maintaining high accuracy. Therefore, transfer learning is widely used in developing practical and scalable medical image analysis systems.

2.1 TECHNOLOGIES USED

Table 2.1 Technologies Used for Squint Eye Disease Detection

Component	Details
Programming language	Python- Primary Language for machine learning tasks.
Libraries	OpenCV, TensorFlow/Keras, NumPy, Scikit-learn
Image Processing	Used to detect facial features and eye regions.
Machine Learning	Used to classify normal vs squint eyes (SVM, Random Forest).
Deep Learning	Used for automatic feature extraction (CNN).

3.SYSTEM ANALYSIS

3.1 Existing System and Limitations

The traditional diagnosis of Strabismus primarily relies on clinical examinations conducted by trained ophthalmologists. These procedures require specialized equipment and professional expertise to accurately evaluate eye alignment. One of the most commonly used methods is the cover test, in which one eye is covered while the patient focuses on a fixed object. The uncovered eye is carefully observed to check whether it shifts position to refocus, indicating possible misalignment. Another widely used technique is the Hirschberg test, where a light is directed into the patient's eyes and the reflection on the cornea is examined. The position of this reflection helps determine whether the eyes are properly aligned or deviated.

Despite their effectiveness, these traditional diagnostic methods have several limitations. They require the presence of skilled ophthalmologists who can both perform the tests and interpret the results accurately. The process can be time-consuming, making it less suitable for large-scale screening or quick assessments. Additionally, access to specialized eye care professionals may be limited in rural or underserved regions, creating barriers to early diagnosis. Furthermore, manual examination may sometimes be subject to human error or variability between practitioners, which can affect consistency in diagnosis. These challenges highlight the need for automated, technology-driven solutions to improve accessibility and efficiency in squint eye detection.

3.2 PROBLEM STATEMENT

The problem addressed in this project is the difficulty in early detection of Strabismus, which often remains unnoticed in its initial stages due to the lack of accessible and efficient screening tools. Traditional diagnostic methods rely heavily on manual examination by ophthalmologists, making the process expensive, time-consuming, and dependent on the availability of skilled professionals. This creates a significant challenge, especially in rural and remote areas where access to specialized eye care services is limited. As a result, many cases go undiagnosed until complications such as Amblyopia or permanent vision impairment develop. Therefore, there is a clear need for an automated, AI-based system that can accurately and efficiently detect squint eye disease using simple image inputs. Such a system would enable early screening, reduce dependency on manual diagnosis, and improve accessibility to eye care services.

3.3 PROPOSED SYSTEM & ADVANTAGES

The proposed system is designed to overcome the limitations of traditional diagnostic methods by leveraging advanced deep learning techniques to automatically detect Strabismus. Unlike conventional approaches that rely on manual examination by ophthalmologists, this system introduces an intelligent and automated solution capable of analysing facial images with minimal human intervention. The process begins with capturing or uploading a facial image, after which the system applies computer vision techniques to detect the face and accurately extract the eye regions. These eye regions are then processed and analysed using a trained neural network model that has learned complex patterns from large datasets of normal and misaligned eyes. By examining features such as pupil position, eye symmetry, and alignment, the model determines whether the eyes are properly aligned or exhibit signs of strabismus.

The use of deep learning enables the system to achieve high levels of accuracy and adaptability, as it can generalize across different individuals, lighting conditions, and facial variations. This makes the system more robust compared to traditional rule-based methods. Furthermore, continuous training and updates to the model can further enhance its performance over time, allowing it to handle more complex cases and improve detection reliability.

This automated approach offers several significant advantages. It greatly reduces the dependency on manual examination, thereby saving time and reducing the workload on ophthalmologists. As a result, healthcare professionals can focus more on critical cases rather than routine screening. The system also provides faster diagnostic results, often within seconds, enabling quick identification of potential squint cases and facilitating early intervention. Early detection is crucial in preventing complications such as Amblyopia and long-term vision impairment.

In addition, the system can be easily deployed as a digital screening tool on various platforms, including computers, tablets, and mobile devices. This portability allows users to perform preliminary eye screening from the comfort of their homes or in remote locations without immediate access to medical facilities. Such accessibility is especially beneficial in rural and underserved areas where specialized eye care services are limited.

Overall, the proposed system has strong potential to transform the way eye alignment disorders are detected. By combining speed, accuracy, and accessibility, it enhances the efficiency of screening processes and supports early diagnosis. This not only improves individual patient outcomes but also contributes to broader public health efforts by enabling large-scale, cost-effective screening of Strabismus.

4.SYSTEM REQUIREMENT SPECIFICATIONS

4.1 FUNTIONAL REQUIREMENTS

The system should:

1. Capture Facial Image Input

This is the first step where the system collects the input image. The image can be captured using a camera (webcam/mobile) or uploaded by the user. It is important that the face is clearly visible, especially the eye region, as the entire analysis depends on image quality.

2. Detect Face and Eye Regions

In this stage, the system identifies the face within the image and then locates the eyes. Techniques like face detection algorithms and facial landmark detection are used to accurately isolate the eye regions. This step ensures that only relevant parts of the image are processed further.

3. Extract Eye Alignment Features

After detecting the eyes, the system proceeds to extract important features that are essential for identifying Strabismus. These features include the position of each eye, the exact location of the pupils, the distance and angular relationship between both eyes, and the overall symmetry of eye alignment. By analysing these parameters, the system can effectively determine whether both eyes are properly aligned or if there is any deviation, which is a key indicator of squint eye condition.

4. Classify Eye Condition

The extracted features are fed into a trained machine learning or deep learning model. The model analyses the data and classifies the eye condition into categories such as:

- Normal eye alignment
- Squint

This classification is based on patterns learned during training.

5. Display Detection Result

Once the classification is complete, the system presents the result to the user. It clearly indicates whether the person has normal eyes or shows signs of squint. The output may also include confidence levels or visual indicators for better understanding.

6. Store Diagnostic Data

Finally, the system stores the results and related data (such as images or analysis outcomes) in a database. This helps in:

- Maintaining patient records
- Tracking progress over time
- Supporting further medical analysis

4.2 NON-FUNCTIONAL REQUIREMENTS

Performance:

The system should be capable of providing real-time detection, meaning it must process input images quickly and deliver results without noticeable delay. This is especially important for live applications such as webcam-based screening or mobile usage, where users expect instant feedback. Efficient algorithms and optimized models are required to ensure smooth and fast performance even on devices with limited computational power.

Accuracy:

High classification accuracy is essential for correctly identifying whether a person has Strabismus or normal eye alignment. The system must minimize false positives and false negatives to ensure reliable results. This can be achieved by training the model on a large and diverse dataset, allowing it to generalize well across different faces, lighting conditions, and angles.

Reliability:

The system should consistently produce the same results when given similar inputs, ensuring dependability in real-world usage. It must perform well under various conditions such as different lighting, image quality, and facial orientations. A reliable system builds trust among users and healthcare professionals by maintaining stable and predictable performance over time.

Usability:

A user-friendly interface is crucial to make the system accessible to both medical professionals and general users. The application should be easy to navigate, with clear instructions for capturing or uploading images and understanding results. Simple design, intuitive controls, and readable outputs enhance user experience and encourage wider adoption.

Scalability:

The system should be designed to handle multiple images efficiently, making it suitable for large-scale screening programs. It should be capable of processing a high volume of data without significant performance degradation. Scalability ensures that the system can be deployed in hospitals, clinics, or public health campaigns where many users need to be screened quickly and effectively.

4.3 SOFTWARE REQUIREMENTS

- Operating System: Windows/Linux
- Programming Language: Python
- Tools: Jupyter Notebook / VS Code
- Libraries:
 - OpenCV
 - TensorFlow
 - Scikit-learn
 - NumPy

4.4 HARDWARE REQUIREMENTS

- Processor: Intel i3 or above
- RAM: Minimum 4GB (8GB recommended)
- Camera/Webcam
- Hard Disk: 200GB free space

5.SYSTEM DESIGN

5.1 SYSTEM SPECIFICATIONS

The Squint Eye Disease Detection System is designed as a computer vision–based application that utilizes deep learning techniques to identify strabismus from facial images. The system requires a standard computing environment with moderate processing capabilities to ensure smooth execution of both training and prediction tasks. A system with at least an Intel Core i5 processor, 8 GB RAM, and sufficient storage is recommended. For improved performance, especially during model training, the use of a GPU-enabled system is advantageous.

From a software perspective, the system is developed using Python as the primary programming language due to its extensive support for machine learning and image processing libraries. Frameworks such as TensorFlow and Keras are employed for designing and training the deep learning model. Image processing and manipulation are carried out using OpenCV, while facial landmark detection is performed using MediaPipe. These tools collectively enable efficient detection and analysis of eye regions.

The system follows a structured architecture consisting of multiple modules that work sequentially. Initially, the system accepts an input image containing a human face. The face detection module identifies key facial landmarks, from which the eye regions are extracted. The extracted regions are then passed through a preprocessing stage where operations such as resizing, normalization, and augmentation are applied to standardize the data.

5.2 SYSTEM COMPONENTS

In this application there are several components as follows

Module 1: Performance Module

The Performance Module focuses on ensuring that the system operates with high speed and efficiency. It is responsible for enabling real-time or near real-time detection, which is essential for applications such as live camera analysis or mobile-based screening. This module optimizes various stages of the system, including image acquisition, preprocessing, face and eye detection, and model execution. It also ensures efficient utilization of computational resources such as CPU and memory so that the system can run smoothly even on low-end devices. A well-optimized performance module reduces delays and provides users with quick results, improving overall user experience.

Module 2: Accuracy Module

The Accuracy Module is responsible for delivering precise and correct classification results. It ensures that the system can reliably detect Strabismus while minimizing errors such as false positives and false negatives. This module involves training the machine learning or deep learning model on a large and diverse dataset that includes various eye conditions, lighting environments, and facial variations. Techniques such as data augmentation, model validation, and hyperparameter tuning are used to enhance performance. High accuracy is crucial because incorrect predictions can lead to misinterpretation and reduce the effectiveness of the system as a diagnostic aid.

Module 3: Reliability Module

The Reliability Module ensures that the system provides consistent and stable outputs across different operating conditions. It focuses on making the system robust against variations such as lighting changes, image resolution differences, head tilts, and facial expressions. The system should produce similar results for similar inputs, ensuring repeatability and trustworthiness. This module also includes error handling mechanisms to manage unexpected inputs or system failures gracefully. A reliable system is essential for building user confidence and ensuring that the tool can be used repeatedly without inconsistency.

Module 4: Usability Module

The Usability Module is designed to enhance the interaction between the user and the system. It ensures that the interface is simple, intuitive, and easy to navigate, even for users with minimal technical knowledge. This includes clear instructions for capturing or uploading images, guided workflows, and easy-to-understand output results. The module may also include visual aids, alerts, or messages to assist users throughout the process. By focusing on user experience, this module ensures that the system can be effectively used in various environments such as clinics, homes, and rural healthcare centres.

Module 5: Scalability Module

The Scalability Module ensures that the system can handle increasing workloads efficiently. It allows the system to process multiple images or serve multiple users simultaneously without slowing down or losing performance. This is particularly important for large-scale applications such as hospital screenings, school health programs, or public health initiatives. The module may involve the use of cloud-based infrastructure, parallel processing techniques, and efficient data storage systems. Scalability ensures that the system can expand as demand grows, making it suitable for widespread deployment and long-term use.

5.3 UML DIAGRAMS

Unified Modelling Language (UML) is a standardized modelling language widely used in software engineering to specify, visualize, construct, and document the various components and processes of a system. It provides a structured and systematic way of representing both the static structure and dynamic behaviour of a system through a set of well-defined graphical notations. UML plays a crucial role in simplifying the design and development of complex systems by breaking them down into understandable visual representations. These diagrams help developers, designers, and stakeholders to clearly interpret how a system is organized and how different components interact with each other.

One of the major advantages of UML is its ability to support multiple types of diagrams, each serving a specific purpose in system design and analysis. Common UML diagrams include use case diagrams, class diagrams, sequence diagrams, and activity diagrams. Use case diagrams describe the interactions between users and the system, highlighting the functionalities provided. Class diagrams represent the structure of the system by showing classes, attributes, methods, and relationships. Sequence diagrams illustrate the flow of messages between different components over time, while activity diagrams depict the workflow or sequence of operations within the system. Together, these diagrams provide a comprehensive understanding of both the functional and structural aspects of the system.

In the context of this project, UML plays an important role in representing the architecture and workflow of the squint eye detection system. It helps in clearly visualizing how different components such as image input, preprocessing modules, feature extraction units, and machine learning models interact with each other. By using UML diagrams, the entire process—from uploading a facial image to generating the final prediction—can be represented in a clear and organized manner. This not only improves the understanding of the system but also helps in identifying the sequence of operations involved in detecting eye misalignment.

Another key benefit of UML is that it facilitates effective communication among stakeholders, including developers, project managers, and non-technical users. Visual representations are easier to understand compared to complex textual descriptions, making it possible for all stakeholders to participate in discussions and decision-making processes. This ensures that the system requirements are clearly defined and understood by everyone involved in the project.

Furthermore, UML helps in identifying potential design issues at an early stage of development. By analysing diagrams, developers can detect inconsistencies, missing components, or inefficient workflows before the actual implementation begins. This reduces the chances of errors and minimizes the need for major changes during later stages of development. As a result, the overall development process becomes more efficient and cost-effective.

In addition, UML supports maintaining a clear and consistent design throughout the software development lifecycle. From initial requirement analysis to final implementation and maintenance, UML diagrams serve as a reference that guides the development process. They help ensure that the system is built according to the planned design and that all components function as intended. This consistency is especially important in projects involving multiple developers or complex architectures.

Overall, the use of UML in this project enhances clarity, improves communication, and ensures a well-structured approach to system design. It provides a strong foundation for developing a reliable and efficient squint eye detection system by offering clear insights into system architecture, component interactions, and workflow processes. By integrating UML into the design phase, the project achieves better organization, reduced complexity, and improved overall quality

Advantages

- UML diagrams provide a visual language that enhances understanding among stakeholders, bridging gaps between technical and non-technical participants.
- They offer a clear representation of system architecture, making complex interactions easier to comprehend and analyze during the design phase.
- UML is a widely recognized standard, ensuring consistency in design representation, which simplifies collaboration among different teams and stakeholders.
- Diagrams serve as essential documentation, helping to maintain clarity on system functionality, design decisions, and interactions throughout the development process.
- UML aids in gathering and validating requirements by illustrating how components interact, ensuring that all needs are addressed effectively.

Disadvantages

- Some UML diagrams can become overly complex, making it challenging for stakeholders to grasp essential concepts without adequate explanation.
- Understanding UML notation and its various diagrams may require additional training for team members unfamiliar with the language.
- Creating comprehensive UML diagrams can be time-intensive, potentially delaying the overall development process if not managed effectively.
- Maintaining and updating UML diagrams can introduce overhead, especially when system requirements evolve frequently during the development lifecycle.
- UML may not capture all nuances of certain systems or behaviors, necessitating supplementary documentation or tools for a complete understanding.

5.3.1 CLASS DIAGRAM

A class diagram for the Squint Eye Disease Detection System represents the static structure of the application by illustrating the main classes involved in image processing and classification, along with their attributes, methods, and relationships. Using the principles of Unified Modelling Language, the system can be modelled with classes such as Image Input, Face Detector, Eye Extractor, Preprocessor, Model (based on MobileNetV2), and Prediction System. The Image Input class handles the input of facial images, which are then processed by the Face Detector class using Media Pipe to identify facial landmarks. The Eye Extractor class extracts the eye regions based on these landmarks, and the Preprocessor class performs operations.

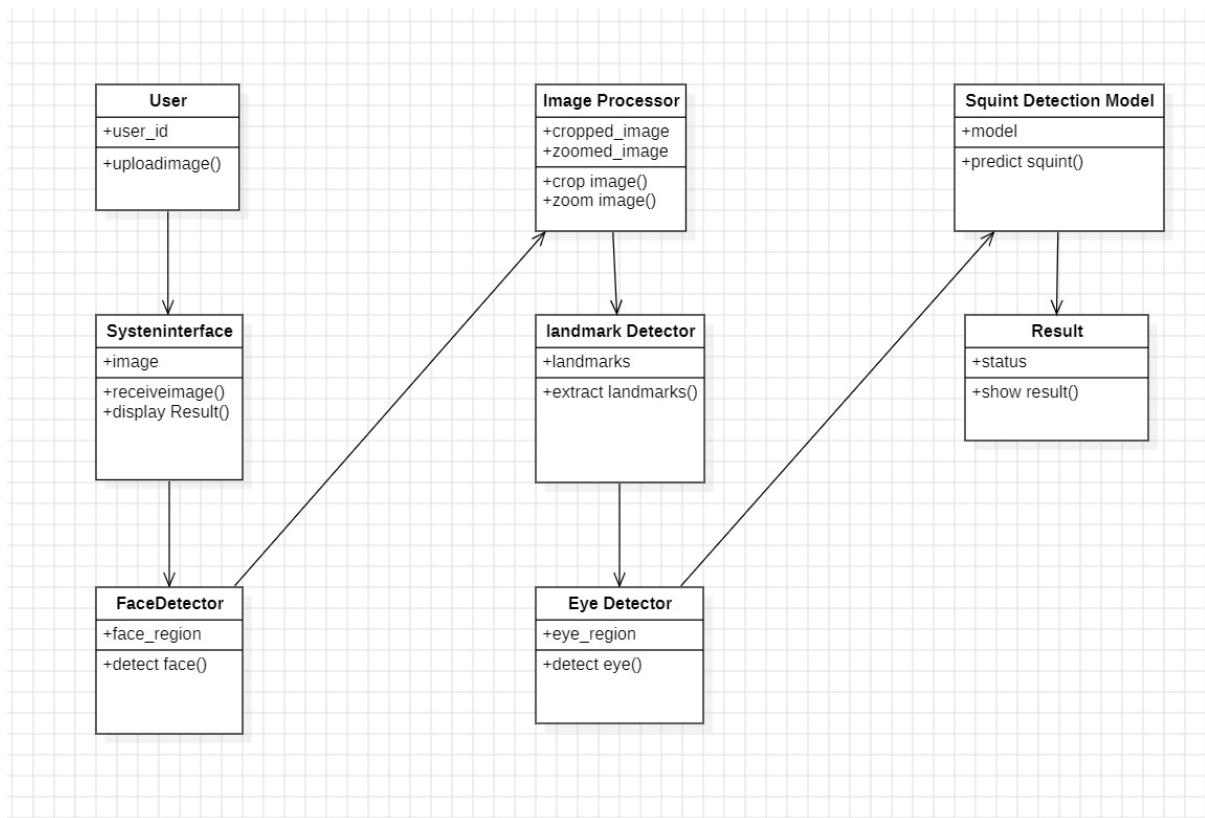


Figure 5.1: Class Diagram for Squint Eye Disease Detection

5.3.2 USECASE DIAGRAM

A usecase diagram for the Squint Eye Disease Detection System describes how users interact with the system to achieve specific goals. It is a part of the Unified Modelling Language and focuses on the functional behaviour of the system from the user's perspective. The primary actor in this system is the user, who interacts with the application to upload or capture a facial image. The main use cases include image input, eye detection, preprocessing, model prediction, and result display. When the user provides an image, the system automatically detects facial landmarks using MediaPipe, extracts the eye region, and preprocesses the image.

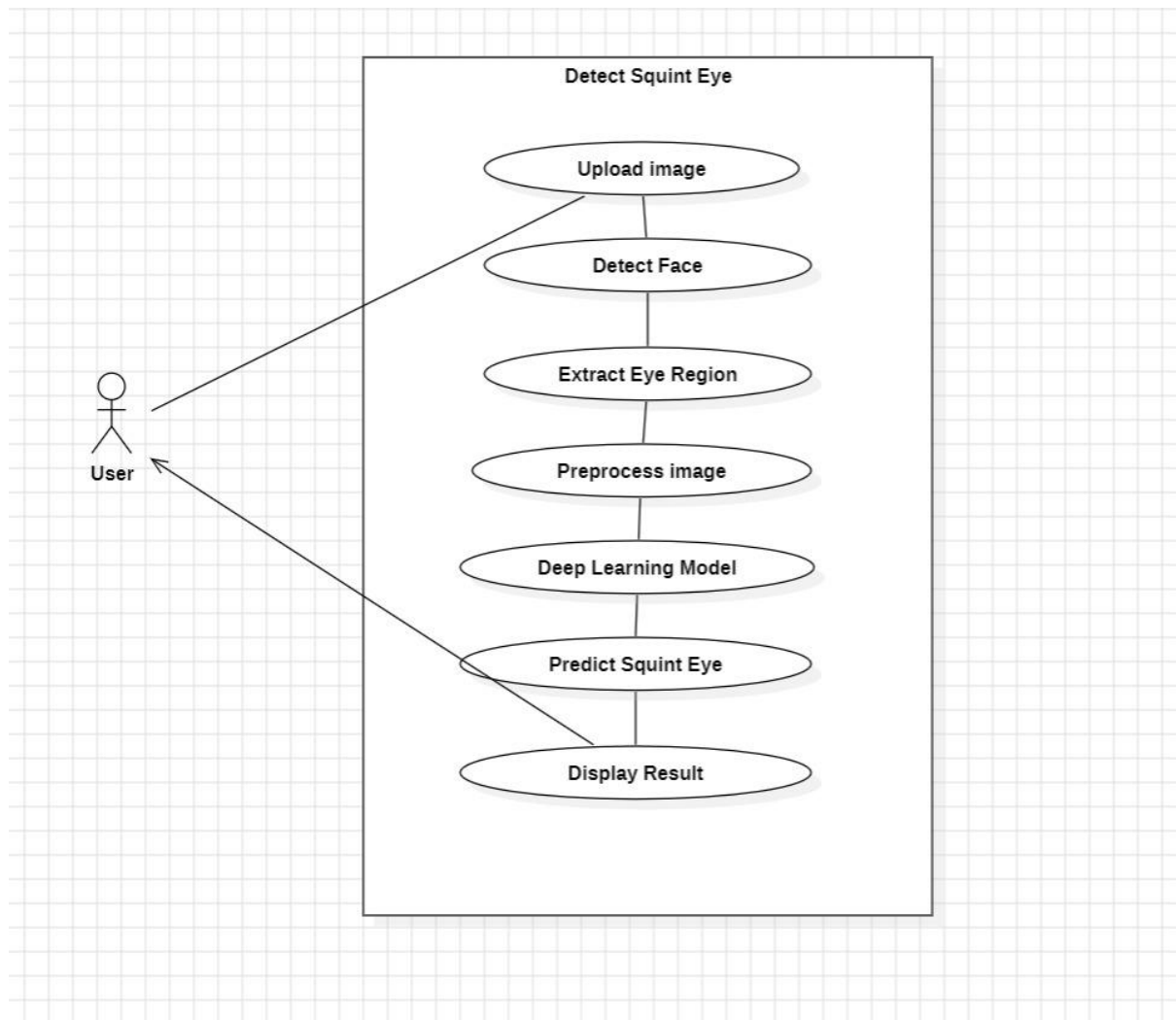


Figure 5.2: Usecase Diagram for Squint Eye Disease Detection

5.3.3 ACTIVITY DIAGRAM

An activity diagram for the Squint Eye Disease Detection System represents the step-by-step workflow of the system, illustrating how different processes are executed from input to output. It is a behavioural diagram in the Unified Modelling Language that focuses on the flow of activities and decision-making within the system. The process begins with the user providing an input image, after which the system performs face detection and identifies facial landmarks using MediaPipe. The eye regions are then extracted and passed through preprocessing steps such as resizing, normalization, and augmentation.

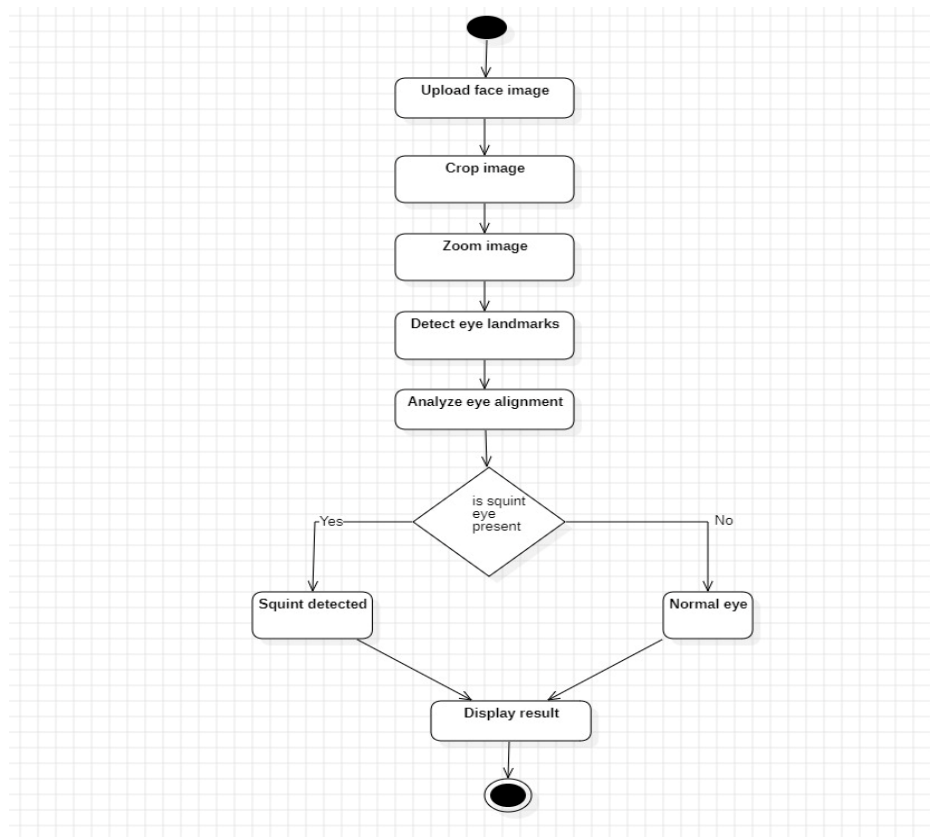


Figure 5.3: Activity Diagram for Squint Eye Disease Detection

5.3.4 COMPONENT DIAGRAM

This component diagram represents the architecture of a churn prediction system. It begins with the UI module, which interacts with users and passes requests through the API Gateway. The Authentication Service verifies user access, and the Reporting Module provides results and insights. The Prediction Engine handles requests for churn prediction, using data processed through Feature Engineering and Data Preprocessing, which retrieve and prepare data from the Customer Data DB. The Model Trainer builds models, which are then stored in the Model Store. These models are accessed by the Prediction Engine for making predictions. The components are modular, enabling easy maintenance and scalability of the system.

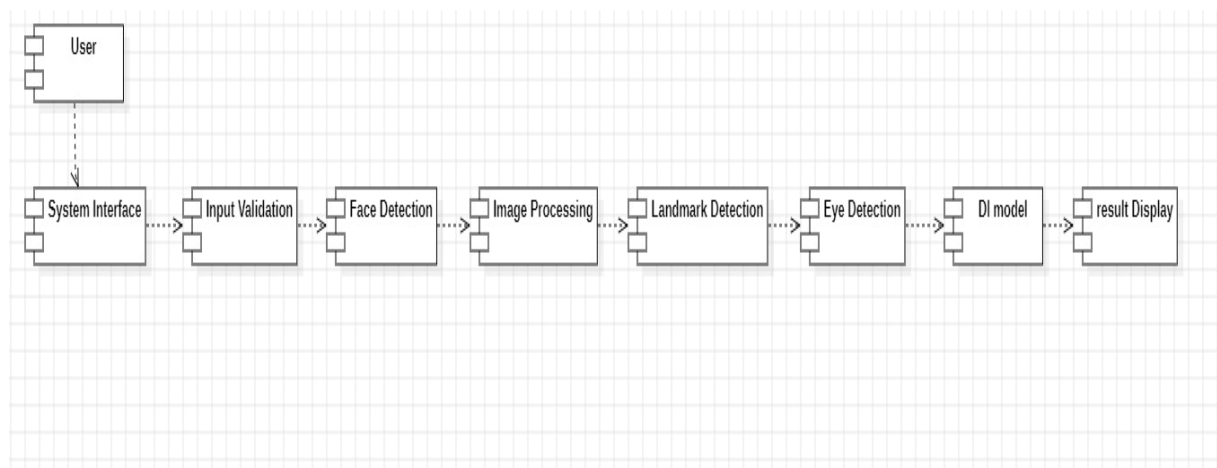


Figure 5.4: Component Diagram for Squint Eye Disease Detection

5.3.5 SEQUENCE DIAGRAM

A sequence diagram for the Squint Eye Disease Detection System illustrates the dynamic interaction between different system components over time, showing how data flows from the user to the final prediction. It is a type of interaction diagram in the Unified Modeling Language that emphasizes the order of messages exchanged between objects. The sequence begins when the user uploads or captures an image, which is sent to the Image Input module. The image is then passed to the Face Detection component, where facial landmarks are identified using MediaPipe.

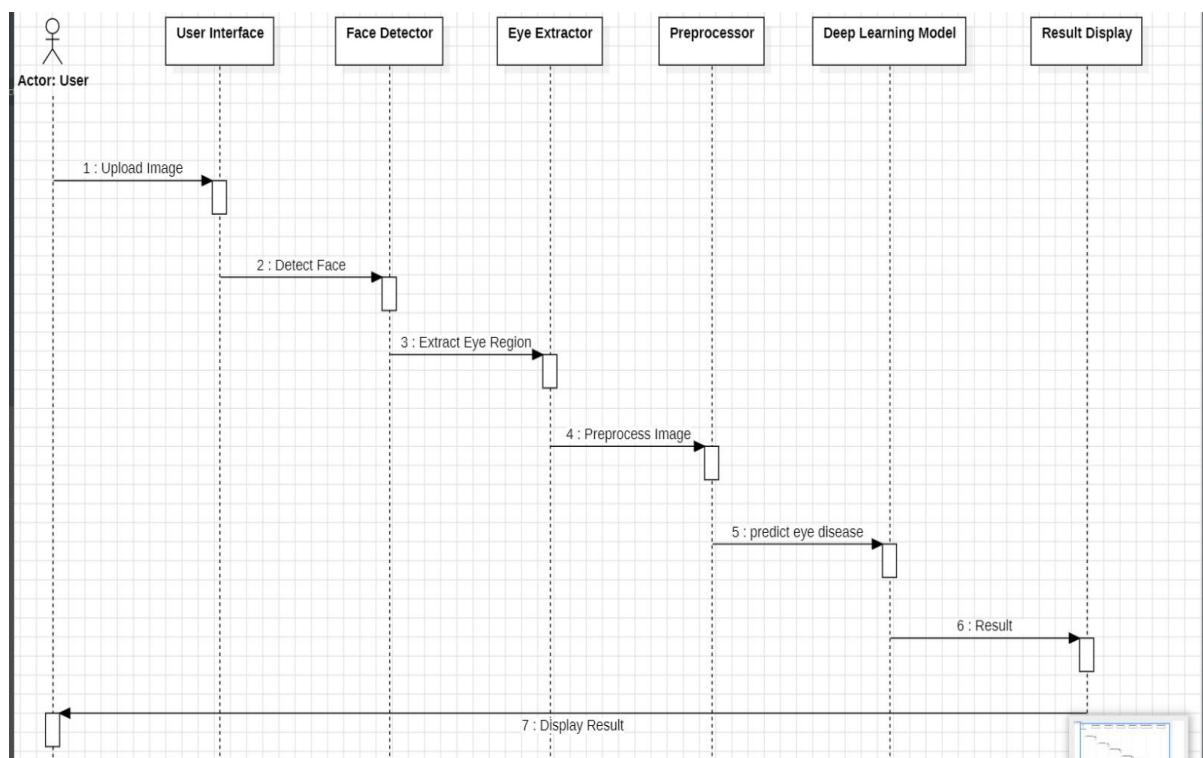


Figure 5.5: Sequence Diagram for Squint Eye Disease Detection

6.IMPLEMENTATION

6.1 SAMPLE CODE

```
import os
import shutil # os and shutil for file handling
import random # random image sampling
import tensorflow as tf # deep learning
import numpy as np
SEED = 42
tf.random.set_seed(SEED)
np.random.seed(SEED)
random.seed(SEED)
os.environ["PYTHONHASHSEED"] = str(SEED)
!pip install mediapipe
import mediapipe as mp
from tqdm import tqdm
from mediapipe.tasks import python
from mediapipe.tasks.python import vision
base_options = python.BaseOptions(
    model_asset_path="/kaggle/input/models/haikalthrq/mp-face-landmarker/other/default/1/face_landmarker.task"
)
options = vision.FaceLandmarkerOptions(
    base_options=base_options,
    output_face_blendshapes=False,
    output_facial_transformation_matrixes=False,
    num_faces=1
)
face_mesh = vision.FaceLandmarker.create_from_options(options)
def extract_eye_region(image):
    h, w, _ = image.shape
    mp_image = mp.Image(
        image_format=mp.ImageFormat.SRGB,
        data=cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```

)
results = face_mesh.detect(mp_image)
# No face detected
if len(results.face_landmarks) == 0:
    return None
landmarks = results.face_landmarks[0]
# Eye corner landmarks
LEFT_EYE = [33, 133]
RIGHT_EYE = [362, 263]
x_coords = []
y_coords = []
for idx in LEFT_EYE + RIGHT_EYE:
    x = int(landmarks[idx].x * w)
    y = int(landmarks[idx].y * h)
    x_coords.append(x)
    y_coords.append(y)
# ----- POSE FILTER 1: YAW (side turn) -----
nose_x = landmarks[1].x * w
left_eye_x = landmarks[33].x * w
right_eye_x = landmarks[263].x * w
dist_left = abs(nose_x - left_eye_x)
dist_right = abs(nose_x - right_eye_x)
if dist_right == 0:
    return None
yaw_ratio = dist_left / dist_right
if not (0.5 < yaw_ratio < 1.5):
    return None
# ----- POSE FILTER 2: ROLL (tilt) -----
left_eye_y = landmarks[33].y * h
right_eye_y = landmarks[263].y * h
if abs(left_eye_y - right_eye_y) > 20:
    return None

# ----- CROP -----

```

```

x_min, x_max = min(x_coords), max(x_coords)
y_min, y_max = min(y_coords), max(y_coords)
pad = 20
x_min = max(0, x_min - pad)
y_min = max(0, y_min - pad)
x_max = min(w, x_max + pad)
y_max = min(h, y_max + pad)
cropped = image[y_min:y_max, x_min:x_max]
return cropped

import cv2
img =
cv2.imread("/kaggle/input/datasets/druthvikvarma/strabismusdatasarchive(5)/utkcr
opped/100_0_0_20170112213500903.jpg.chip.jpg")
crop = extract_eye_region(img)
print("Success" if crop is not None else "Failed")
OUTPUT_DIR = "/kaggle/working/eye_dataset"
splits = ["train", "val", "test"]
classes = ["normal", "strabismus"]
for split in splits:
    for cls in classes:
        os.makedirs(os.path.join(OUTPUT_DIR, split, cls), exist_ok=True)
BASE_STRABISMUS="/kaggle/input/datasets/druthvikvarma/strabismus-
dataset/archive (2)/STRABISMUS"
BASE_UTK = "/kaggle/input/datasets/druthvikvarma/strabismus-dataset/archive
(5)/utkcropped"
strabismus_classes = ["ESOTROPIA", "EXOTROPIA", "HYPERTROPIA",
"HYPOTROPIA"]
normal_medical_class = "NORMAL"
all_images = []
for cls in strabismus_classes:
    class_path = os.path.join(BASE_STRABISMUS, cls)
    for img in os.listdir(class_path):
        all_images.append(os.path.join(class_path, img))
random.shuffle(all_images)

```

```

train_split = int(0.7 * len(all_images))
val_split = int(0.85 * len(all_images))
train_imgs = all_images[:train_split]
val_imgs = all_images[train_split:val_split]
test_imgs = all_images[val_split:]
def copy_images(image_list, split):
    count = 0
    for img_path in tqdm(image_list):
        img_name = os.path.basename(img_path)
        dst = os.path.join(OUTPUT_DIR, split, "strabismus", img_name)
        shutil.copy(img_path, dst)
        count += 1
    print(f"{split} copied:", count)
copy_images(train_imgs, "train")
copy_images(val_imgs, "val")
copy_images(test_imgs, "test")
medical_normal_path = os.path.join(BASE_STRABISMUS,
normal_medical_class)
all_normal_medical = {
    os.path.join(medical_normal_path, img)
    for img in os.listdir(medical_normal_path)
}
random.shuffle(all_normal_medical)
train_split = int(0.7 * len(all_normal_medical))
val_split = int(0.85 * len(all_normal_medical))
train_nm = all_normal_medical[:train_split]
val_nm = all_normal_medical[train_split:val_split]
test_nm = all_normal_medical[val_split:]
def copy_normal_medical(image_list, split):
    count = 0
    for img_path in image_list:
        img_name = os.path.basename(img_path)
        dst = os.path.join(OUTPUT_DIR, split, "normal", img_name)
        shutil.copy(img_path, dst)

```

```

        count += 1

    print(f"Medical NORMAL {split} copied:", count)
copy_normal_medical(train_nm, "train")
copy_normal_medical(val_nm, "val")
copy_normal_medical(test_nm, "test")
random.seed(SEED)
TARGET_UTK = 370
utk_root = BASE_UTK # this points to utkcropped
all_utm_images = {
    os.path.join(utk_root, img)
    for img in os.listdir(utk_root)
}
random.shuffle(all_utm_images)
selected_utm = all_utm_images[: TARGET_UTK]
# SPLIT 70 / 15 / 15
train_u_split = int(0.7 * len(selected_utm))
val_u_split = int(0.85 * len(selected_utm))
train_utm = selected_utm[:train_u_split]
val_utm = selected_utm[train_u_split:val_u_split]
test_utm = selected_utm[val_u_split:]
def save_utm(image_list, split):
    saved = 0
    output_dir = os.path.join(OUTPUT_DIR, split, "normal")
    for img_path in tqdm(image_list):
        image = cv2.imread(img_path)
        if image is None:
            continue
        cropped = extract_eye_region(image)
        if cropped is not None:
            img_name = "utm_" + os.path.basename(img_path)
            save_path = os.path.join(output_dir, img_name)
            cv2.imwrite(save_path, cropped)
            saved += 1
    print(f"UTK {split} saved:", saved)

```

```

save_utm(train_utm, "train")
save_utm(val_utm, "val")
save_utm(test_utm, "test")
for split in ["train", "val", "test"]:
    print("\n", split)
    print("NORMAL:",len(os.listdir(f'/kaggle/working/eye_dataset/{split}/normal')))
)
    print("STRABISMUS:",len(os.listdir(f'/kaggle/working/eye_dataset/{split}/strab
ismus'))))
train_path = "/kaggle/working/eye_dataset/train"
val_path  = "/kaggle/working/eye_dataset/val"
test_path = "/kaggle/working/eye_dataset/test"
from tensorflow.keras.preprocessing.image import ImageDataGenerator # used for
real-time augmentation and preprocessing of image data
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=15,
    zoom_range=0.1,
    horizontal_flip=True
)
val_test_datagen = ImageDataGenerator(rescale=1./255)
train_data = train_datagen.flow_from_directory(
    train_path,
    target_size = (224,224),
    batch_size = 32,
    class_mode = 'binary',
    shuffle = True,
    seed = SEED
)
val_data = val_test_datagen.flow_from_directory(
    val_path,
    target_size = (224,224),
    batch_size = 32,
    class_mode = 'binary',

```

```

    shuffle = False,
)
test_data = val_test_datagen.flow_from_directory(
    test_path,
    target_size = (224,224),
    batch_size = 32,
    class_mode = 'binary',
    shuffle = False
)
from tensorflow.keras.applications import MobileNetV2 # transfer learning
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D # dense is
used for predictions & GAP2D summarizes image feature map into a single value
from tensorflow.keras.models import Model
base_model = MobileNetV2(weights='imagenet', include_top=False,
input_shape=(224,224,3)) #uses pretrained feature extractor
for layer in base_model.layers:
    layer.trainable = False # used because small dataset, prevents overfitting and
faster training
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x) # converts extracted features into binary
prediction
output = Dense(1, activation='sigmoid')(x) # binary classification
model = Model(inputs=base_model.input, outputs=output)
model.compile(
    optimizer = 'adam', # updates model weights based on training
    loss = 'binary_crossentropy', # two classes, normal vs strabismus
    metrics = ['accuracy',
tf.keras.metrics.Recall(name="recall"),tf.keras.metrics.AUC(name="auc")])
from tensorflow.keras.callbacks import EarlyStopping # stops when validation stops
growing, prevents overfitting
from tensorflow.keras.callbacks import ModelCheckpoint # saves best model
callbacks = {

```

```

EarlyStopping(monitor="val_recall", mode="max", patience=6,
restore_best_weights=True, verbose=1),
ModelCheckpoint("best_model.keras", monitor="val_recall", mode="max",
save_best_only=True, verbose=1)
}
from sklearn.utils.class_weight import compute_class_weight
import numpy as np
classes = np.array([0, 1])
class_weights = compute_class_weight(
    class_weight = "balanced",
    classes = classes,
    y = train_data.classes
)
class_weights = dict(enumerate(class_weights))
print(class_weights)
EPOCHS = 40
history = model.fit(
    train_data,
    validation_data = val_data,
    epochs = EPOCHS,
    class_weight = class_weights,
    callbacksn = callbacks
)
model.evaluate(test_data)
import numpy as np
y_pred = model.predict(test_data)
y_pred = (model.predict(test_data) > 0.35).astype(int)
y_true = test_data.classes
from sklearn.metrics import classification_report, confusion_matrix
print(confusion_matrix(y_true, y_pred))
print(classification_report(
    y_true, y_pred,
    target_names = ["Normal", "Strabismus"]
))

```

```

'# Unfreeze top layers for fine-tuning
for layer in model.layers[-60:]:
    layer.trainable = True
model.compile(
    optimizer = tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss = 'binary_crossentropy',
    metrics = ['accuracy']
)
fine_tune_history = model.fit(
    train_data,
    validation_data = val_data,
    epochs = 5,
    callbacks = callbacks
)
#test_data.reset()
#y_prob = model.predict(test_data, verbose=1)
'''THRESHOLD = 0.35
y_pred = (y_prob > THRESHOLD).astype(int).ravel()'''
#y_true = test_data.classes
'''from sklearn.metrics import classification_report, confusion_matrix
print(confusion_matrix(y_true, y_pred))
print(
    classification_report(
        y_true,
        y_pred,
        target_names = ["Normal", "Strabismus"]
    )
)
import matplotlib.pyplot as plt
plt.figure(figsize = (12,4))
# Accuracy
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Val')

```

```

plt.title('Accuracy')
plt.legend()
# Loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train')
plt.plot(history.history['val_loss'], label='Val')
plt.title('Loss')
plt.legend()
plt.show()
from sklearn.metrics import roc_curve, auc
y_prob = model.predict(test_data)
y_true = test_data.classes
fpr, tpr, _ = roc_curve(y_true, y_prob)
roc_auc = auc(fpr, tpr)
plt.figure()
plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.3f}')
plt.plot([0,1],[0,1], '--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
import seaborn as sns
from sklearn.metrics import confusion_matrix
y_pred = (y_prob > 0.5).astype(int)
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels = ['Normal','Strabismus'],
            yticklabels = ['Normal','Strabismus'])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

```

```
tn, fp, fn, tp = cm.ravel()
sensitivity = tp / (tp + fn)
specificity = tn / (tn + fp)
print("Sensitivity (Recall for Strabismus):", sensitivity)
print("Specificity:", specificity)
model.save("strabismus_mobilenetv2.keras")
pip install streamlit tensorflow opencv-python pillow numpy
model=tf.keras.models.load_model("/kaggle/working/best_model.keras")
model.save("/kaggle/working/final_model.keras")
```

7.SYSTEM TESTING

7.1 TYPES OF TESTING

Table 7.1: Table of testing

Test Type	Description	Tools / Method
Unit Testing	Individual modules such as image loader, preprocessing, and prediction functions are tested separately	Python testing, manual function calls
Integration Testing	Interaction between image processing, eye detection, and model prediction modules is tested	End-to-end testing through UI
System Testing	Complete system is tested with real-time inputs to ensure overall functionality	Full workflow execution
Security Testing	Checks for invalid inputs, file handling, and misuse of upload functionality	Input validation and error handling
Model Testing	Deep learning model performance is evaluated on test dataset	Accuracy, precision, recall metrics
Regression Testing	Ensures new updates do not break existing functionalities	Re-running previous test cases
Usability Testing	User interface, ease of image upload, and result display are verified	Manual testing across browsers
Performance Testing	System response time and processing speed are evaluated	Time measurement for predictions
Boundary Testing	Tests extreme cases like large images, no face, or multiple faces	Edge case inputs
Acceptance Testing	Final validation to check if system meets project requirements	User validation/testing

7.2 TEST CASES

Table 7.2: Table of Test cases

Testcase#1	Priority (H, L): High	
Test Objective	To verify detection of squint eye condition from facial image	
Test Descriptive	The system processes the uploaded face image and checks for squint eye presence	
Requirements verified	Eye region extraction and deep learning model prediction	
Test Environment	Web browser / Python-based application	
Actions	Expected	Actual
Upload image with normal eyes	Normal Eye	Normal Eye
Upload image with squint eyes	Strabismus Eye	Strabismus Eye
Pass: Yes	Conditional pass: No	Fail: No

7.3 DISCUSSION OF RESULTS

The experimental results strongly indicate that deep learning techniques are highly effective in detecting squint eye (strabismus) conditions from facial images. This success is largely attributed to the powerful capabilities of convolutional neural networks (CNNs), which are specifically designed for image analysis tasks. Unlike traditional machine learning approaches that rely on manually crafted features, CNNs automatically learn hierarchical feature representations directly from raw image data. This allows them to capture subtle patterns in eye structure, alignment, and positioning that may not be easily identified through manual methods.

One of the most significant advantages of this approach is the use of transfer learning. Instead of training a deep neural network from scratch—which requires a massive amount of labeled data and computational resources—pretrained models (such as those trained on large-scale image datasets) can be fine-tuned for squint eye detection. This not only reduces the need for extensive training data but also accelerates the training process while maintaining high accuracy and robustness.

In addition, Media Pipe facial landmark detection plays a crucial role in enhancing system performance. By accurately identifying key facial points—particularly around the eyes—it enables precise extraction of eye regions from the input images. This focused analysis ensures that the model concentrates on the most relevant features, thereby improving detection accuracy and reducing noise from irrelevant parts of the image.

By integrating computer vision techniques with deep learning models, the system forms a comprehensive pipeline capable of efficiently analyzing facial images and identifying eye alignment abnormalities. This combination allows for automated, scalable, and potentially real-time detection, which can be valuable in medical screening and diagnostic support systems.

However, despite these promising results, several challenges remain. Variations in lighting conditions can alter image quality and obscure important visual details. Differences in facial expressions may distort eye shape and position, while head orientation (such as tilting or turning) can make it difficult for the model to accurately assess eye alignment. These factors introduce variability that can reduce the reliability of predictions.

Another critical limitation is the availability of large and diverse datasets. Deep learning models perform best when trained on datasets that capture a wide range of variations in age, ethnicity, lighting, and imaging conditions. A lack of such diversity can lead to overfitting and reduced generalization in real-world scenarios. Looking ahead, future research can focus on developing more advanced deep learning architectures, such as attention-based models or hybrid frameworks that combine spatial and temporal information. Expanding datasets with more diverse and annotated samples will also be essential. Additionally, incorporating data augmentation techniques and robust preprocessing methods can help mitigate the impact of environmental variations and improve overall system performance.

In conclusion, while the current approach demonstrates strong potential for accurate and efficient squint eye detection, continued advancements in model design, data collection, and preprocessing strategies will be key to achieving reliable real-world deployment.

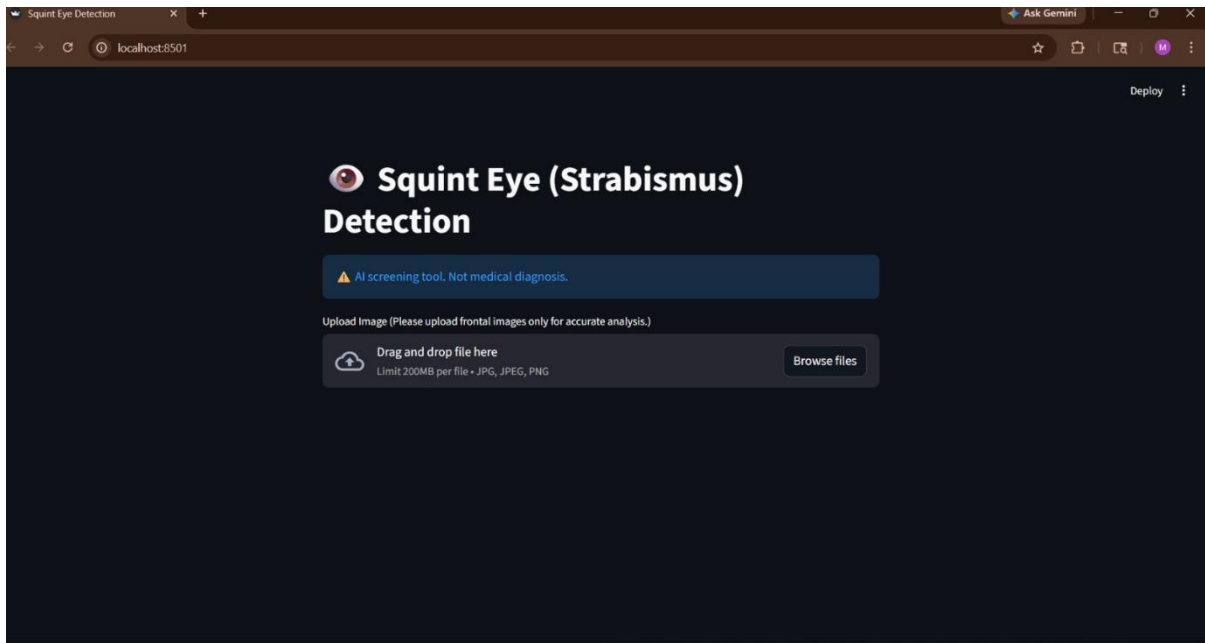


Figure 7.1: Main page for squint eye detection

- The interface is an AI-based web application for detecting squint eye (strabismus).
- It displays the title “Squint Eye (Strabismus) Detection” with an eye icon.
- A warning message states that it is only a screening tool and not a medical diagnosis.
- Users can upload a frontal face image using drag-and-drop or a file browser

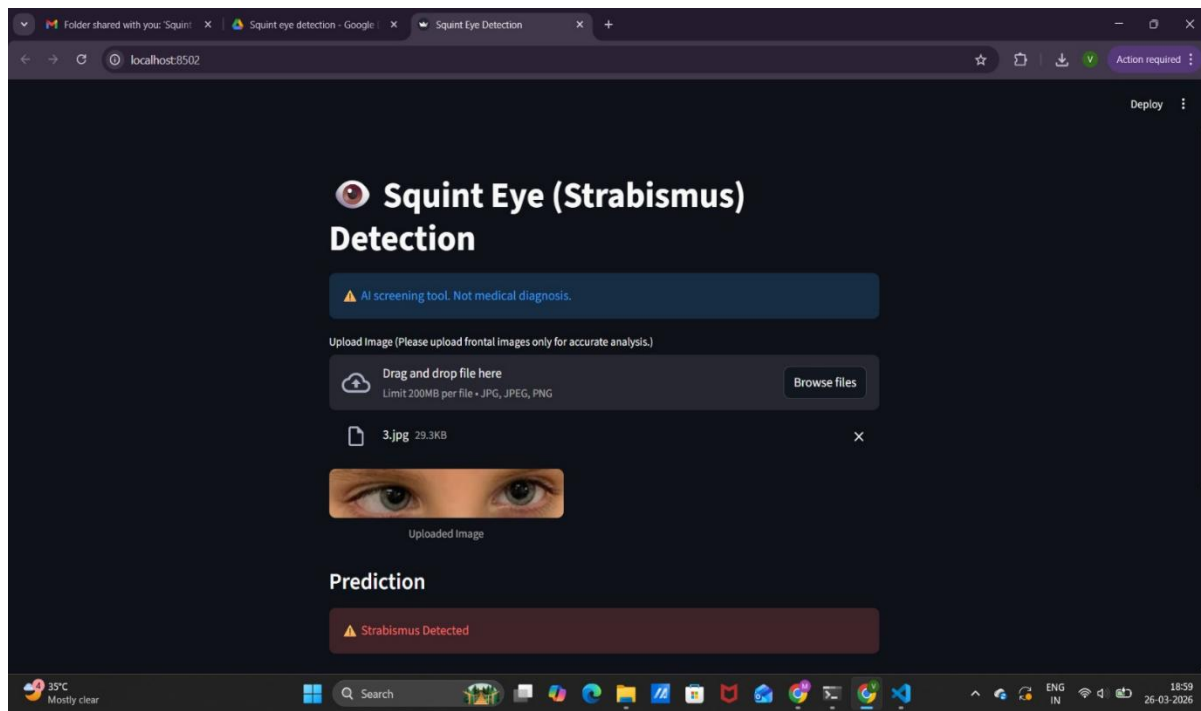


Figure 7.2: Prediction of squint eye

- The interface shows an AI-based application for squint eye (strabismus) detection.
- A warning message indicates it is only a screening tool, not a medical diagnosis.
- An image file has been uploaded, showing a close-up of a person's eyes.
- The uploaded image is displayed below the upload section for preview.
- The system provides a prediction result stating "Strabismus Detected."

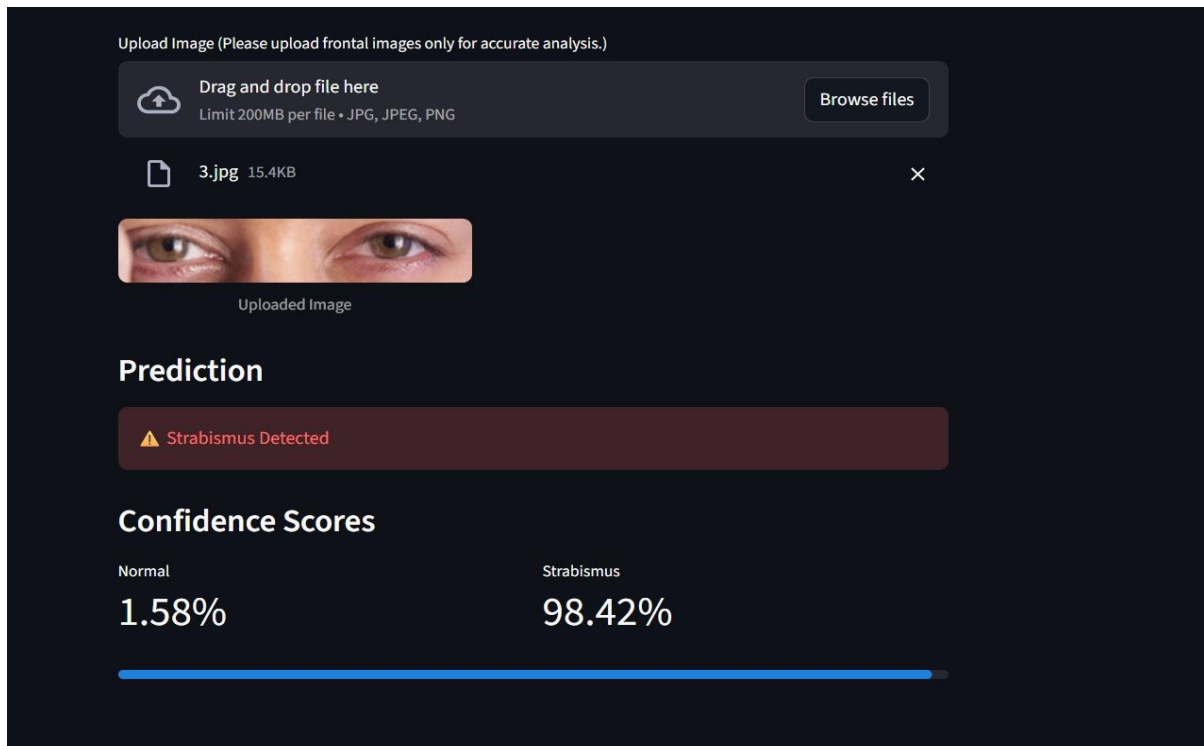


Figure 7.3: Confidence score of squint eyes

- The interface shows an uploaded eye image for analysis.
- The system displays the prediction result as “Strabismus Detected.”
- A confidence score section provides probability values for each class.
- The model predicts Normal: 1.58% and Strabismus: 98.42%.
- A progress bar visually represents the high confidence toward strabismus detection.

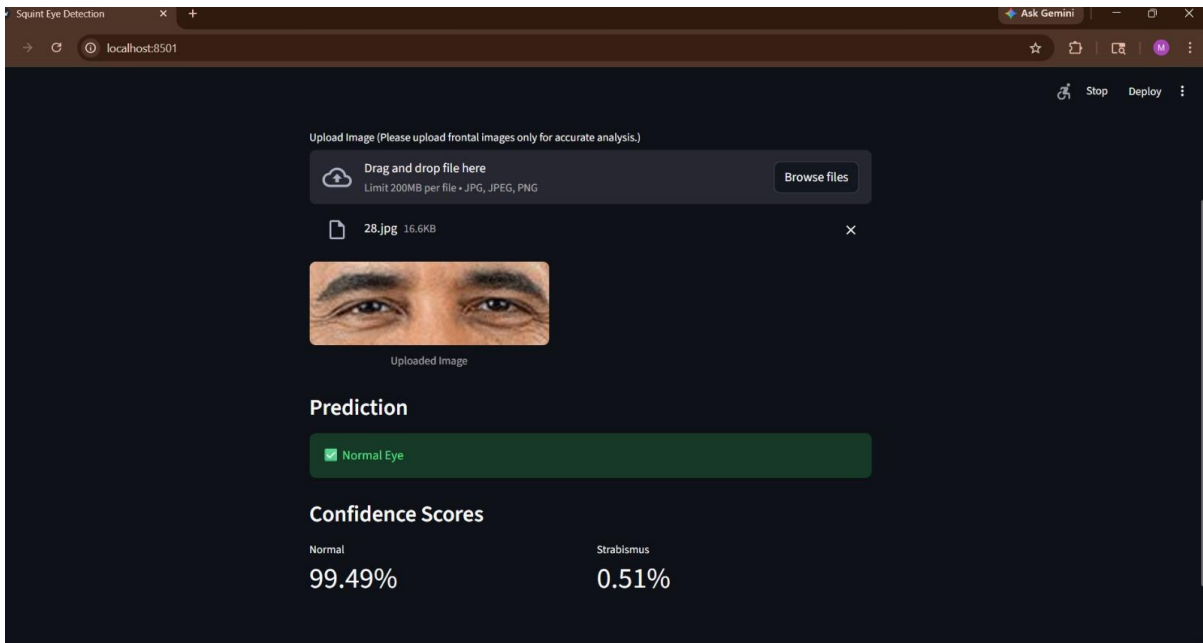


Figure 7.4: Detection of Normal Eye

- The image shows a squint eye detection system interface after uploading a facial image.
- The uploaded image displays a close-up of a person's eyes for analysis.
- The system predicts the result as "Normal Eye."
- Confidence scores are shown with 99.49% for Normal and 0.51% for Strabismus.

8. CONCLUSION & FUTURE ENHANCEMENTS

8.1 CONCLUSION

The experimental results clearly demonstrate that deep learning techniques are highly effective for detecting squint eye (strabismus) conditions from facial images. This effectiveness is primarily due to the capabilities of convolutional neural networks (CNNs), which can automatically learn and extract meaningful features from images without the need for manual feature engineering. Unlike traditional approaches, CNNs identify complex patterns such as eye alignment, shape, and spatial relationships, making them particularly suitable for medical image analysis tasks.

Another important advantage of this approach is the use of transfer learning. By utilizing pretrained models that have already learned general image features from large datasets, the system can be fine-tuned for squint eye detection with a relatively smaller dataset. This significantly reduces the amount of training data and computational resources required, while still achieving high accuracy and reliable performance.

Furthermore, MediaPipe facial landmark detection plays a crucial role in improving the system's precision. It enables accurate identification of facial key points, particularly around the eye region, allowing the model to focus on the most relevant areas of the face. This targeted extraction of eye regions enhances the efficiency and accuracy of the detection process.

By combining computer vision techniques with deep learning models, the system forms an integrated pipeline capable of analysing facial images and detecting eye alignment abnormalities efficiently. This approach not only automates the detection process but also has the potential for real-time applications in medical screening and diagnostic assistance.

However, despite these advantages, several challenges remain. Variations in lighting conditions can affect image quality, while differences in facial expressions and head orientation may distort eye features, leading to reduced detection accuracy. Additionally, the availability of large and diverse datasets is crucial for training robust models. Without sufficient diversity in the training data, the model may struggle to generalize well to real-world scenarios.

Future research can focus on exploring more advanced deep learning architectures, such as attention-based models or hybrid approaches, to further enhance detection performance. Increasing the size and diversity of datasets, along with improved preprocessing and data augmentation techniques, can also contribute to achieving higher accuracy and reliability.

8.2 FUTURE ENHANCEMENTS

The experimental results demonstrate that deep learning techniques are effective for detecting squint eye conditions from facial images. One of the key advantages of using convolutional neural networks (CNNs) is their ability to automatically learn relevant features from images without the need for manual feature engineering. This makes them highly suitable for analyzing complex visual patterns such as eye alignment.

The use of transfer learning further enhances the efficiency of the system by significantly reducing the amount of training data required while still achieving high accuracy. By leveraging pretrained models, the system can adapt to the specific task of squint eye detection with minimal additional training.

In addition, Media Pipe facial landmark detection plays an important role in accurately extracting eye regions from facial images. It helps in identifying precise facial key points, enabling the model to focus on the most relevant areas for analysis. By combining computer vision techniques with deep learning models, the system is able to efficiently analyze facial images and detect eye alignment abnormalities.

However, there are several challenges associated with this approach. Variations in lighting conditions, facial expressions, and head orientation can negatively impact the accuracy of eye detection. Moreover, the availability of large and diverse datasets is essential for improving model performance and ensuring better generalization.

Future research can focus on exploring more advanced deep learning architectures and expanding datasets to further improve accuracy and robustness.

9.REFERENCES

- [1] Software Engineering, A practitioner's Approach- Roger S. Pressman, 6th edition, McGraw Hill International Edition.
- [2] Stephen Marsland, —Machine Learning – An Algorithmic Perspective, Second Edition, Chapman and Hall/CRC Machine Learning and Pattern Recognition Series, 2014.
- [3] Concepts of Programming Languages Robert. W. Sebesta 10/E, Pearson Education.
- [4] M. Yarkheir-Automated Strabismus Detection and Classification using Deep Learning (2025).
- [5] D. Wu et al-Integrating Artificial Intelligence in Strabismus (2024).